

מדינת ישראל - משרד החינוך
שם בית הספר: [הכנס שם בית ספר כאן]

OG P2P Sharer

מערכת שיתוף קבצים מבוזרת (P2P) מאובטחת

במסגרת חלופת "הגנת סייבר ומערכות הפעלה"

מוגש על ידי: [שם התלמיד]

ת.ז: [תעודת זהות]

מנחה הפרויקט: [שם המנחה]

תאריך הגשה: 14/05/2026

תוכן עניינים

3	1. מבוא ורקע הפרויקט
3	1.1 הצגת הבעיה
4	1.2 הפתרון המוצע
5	2. מבנה וארכיטקטורה
5	2.1 סביבת הפיתוח והכלים
6	2.2 עץ קבצי המערכת
7	2.3 פרוטוקול התקשורת (UDP & TCP)
9	3. מדריך למשתמש
9	3.1 התקנה והרצה
10	3.2 ממשק ההתחברות (Lobby)
11	3.3 שיתוף והורדת קבצים
13	4. מדריך למפתח (הסבר טכני)
13	4.1 ניהול קבצים (SharedFilesManager)
15	4.2 גילוי עמיתים (Peer Discovery)
18	4.3 מנגנוני אבטחה והצפנה
22	5. סיכום אישי ורפלקציה

פרק 1: מבוא ורקע הפרויקט

תיאור תכולת הספר

בספר פרויקט זה ריכזתי את כל תהליך העבודה שלי על המערכת "OG P2P Sharer" – משלב הרעיון והתכנון ועד לכתיבת הקוד והבדיקות. במסמך אני מסביר איך המערכת עובדת מאחורי הקלעים: בניתי ארכיטקטורת רשת מבוססת לחלוטין (Peer-to-Peer) ואני מפרט כאן על פרוטוקולי התקשורת שכתבתי (UDP לגילוי עמיתים ו-TCP להעברת קבצים) כדי להעביר את הנתונים בצורה מהירה ובטוחה. בנוסף, הוספתי פירוט טכני על מנגנוני ההצפנה ששילבתי (הגנת סייבר) ועל מבנה המחלקות בקוד, ולבסוף צירפתי סיכום ורפלקציה אישית על תהליך הלמידה.

1.1 הצגת הבעיה

בעידן הדיגיטלי של היום, שיתוף קבצים הוא פעולה יומיומית. עם זאת, רוב הפתרונות הקיימים והנפוצים (כמו Google Drive, WhatsApp או Dropbox) מסתמכים על שרתים מרכזיים (Client-Server Architecture). מודל זה יוצר מספר בעיות טכנולוגיות ואבטחתיות מהותיות.

- **תלות בחיבור לאינטרנט ורוחב פס:** כדי להעביר קובץ למחשב שנמצא באותו חדר או על אותה רשת Wi-Fi, המידע נשלח קודם לשרת מרוחק אי שם בעולם, ורק אז יורד חזרה למחשב היעד. תהליך זה איטי ומיותר.
- **פגיעה בפרטיות ואבטחת מידע:** המידע עובר דרך שרתי צד-שלישי שחשופים לפריצות, דליפות מידע, או פגיעה בפרטיות, ללא שליטה מלאה של המשתמש על הנתונים שלו.
- **נקודת כשל בודדת (Single Point of Failure):** אם השרת המרכזי נופל או קורס, הרשת כולה מושבתת ואף משתמש לא יכול לגשת או להעביר קבצים.

1.2 הפתרון המוצע

המערכת שפיתחתי, **OG P2P Sharer**, יעשה פתרון שיתוף קבצים מאובטח, מהיר ופרטי לרשתות מקומיות (LAN). המערכת בנויה בארכיטקטורת P2P מבוססת לחלוטין, ללא שרת מרכזי כלל, תוך שימת דגש חזק על אבטחת סייבר והצפנת הנתונים.

- **ביזור מלא (Decentralization):** כל משתמש (Peer) ברשת מתפקד גם כלקוח וגם כשרת בו זמנית. המערכת סורקת את הרשת המקומית באופן אוטומטי ומוצאת משתמשים אחרים.
- **הורדת "נחיל" (Swarm Downloading):** אם למספר משתמשים יש את אותו הקובץ, המערכת תוריד מקטעים שונים (Chunks) מכל המשתמשים במקביל, מה שמאיץ משמעותית את קצב ההורדה.
- **אבטחת סייבר מתקדמת:** התקשורת כולה מוצפנת. שלב גילוי העמיתים (UDP), מאובטח באמצעות מפתח סימטרי משותף המונע חדירה לרשת הפרטית (Broadcast) תוך הגנה מפני התקפות שחזור (Replay Attacks).
- **ממשק משתמש (Web UI):** המערכת מספקת ממשק ניהול נוח וידידותי מבוסס דפדפן. המציג את הקבצים הזמינים ברשת ומאפשר מעקב נוח אחר הורדות (Flask).

פרק 2: מבנה וארכיטקטורה

בפרק זה אפרט על המבנה הטכני של המערכת, הכלים בהם השתמשתי והאופן שבו רכיבי התוכנה השונים מתקשרים זה עם זה ליצירת רשת מבוצרת אחת.

2.1 סביבת הפיתוח והכלים

הפרויקט פותח בשפת **Python 3** שנבחרה בשל היכולות המובנות שלה לטיפול בתקשורת רשת, הצפנה וריבוי תהליכים (Threading).

- **שפת פיתוח:** Python 3.10+

- **ספריות רשת:**

socket

(לתקשורת UDP/TCP ברמה נמוכה).

- **אבטחת מידע:** ספריית

cryptography

(RSA ו-Fernet).

- **ממשק משתמש (Web UI):**

Flask

ליצירת שרת אינטרנט מקומי.

- **ניהול נתונים:**

json

להעברת הודעות ו-

hashlib

ליצירת Checksums.

עבודה במקביל:

threading

להאזנה להורדות ושידור נוכחות במקביל.

2.2 עץ קבצי המערכת

שם הקובץ	תפקיד במערכת
main.py	נקודת הכניסה לתוכנית. מאתחל את כל הרכיבים (Peer, Server, UI).
peer.py	אחראי על מנגנון ה-Discovery. משדר נוכחות ב-UDP ומנהל את רשימת השכנים.
tcp_server.py	שרת ה-TCP שממתין לבקשות הורדה ושולח מקטעי קבצים.
tcp_client.py	מנהל את תהליך ההורדה, מבקש Chunks ומרכיב את הקובץ.
security.py	לוגיקת הצפנה היברידית (RSA ו-Fernet).

שם הקובץ	תפקיד במערכת
shared_files_manager.py	סריקת תיקיית השיתוף, יצירת Hashing וניהול המטה-דאטה.
web_ui.py	ניהול שרת ה-Flask והניתובים עבור ממשק המשתמש.
protocol.py	הגדרת פורמט ההודעות (Headers, Ports, Buffer Sizes).

2.3 פרוטוקול התקשורת (UDP & TCP)

גילוי עמיתים (UDP Discovery)

כל עמית שולח הודעת **UDP Broadcast** לפורט 50000 כל 5 שניות. ההודעה כוללת את מזהה העמית ורשימת קבציו. יתרון ה-UDP הוא המהירות והיכולת להגיע לכל המחשבים ברשת המקומית ללא IP ידוע מראש.

העברת נתונים מאובטחת (Secure TCP)

ביקשת הורדה יוצרת חיבור **TCP** שיר בפורט 50001. TCP מבטיח אמינות בהעברת המידע. החיבור עובר "לחיצת יד" מוצפנת לפני העברת הקובץ.

פרק 3: מדריך למשתמש

3.1 התקנה והרצה

יש לוודא שמותקנת סביבת Python 3 ולהריץ:

```
pip install flask cryptography
```

מפעילים על ידי הפקודה:

```
python main.py
```

. המערכת תפתח שרת מקומי בכתובת: **http://localhost:5000**.

3.2 ממשק ההתחברות (The Lobby)

בכניסה לאתר, המשתמש נדרש להזין **מפתח רשת (Swarm Password)** רק משתמשים עם אותה סיסמה יראו אחד את השני, מה שמאפשר יצירת חדרים פרטיים על אותה רשת Wi-Fi.

[צילום מסך: מסך ה-Lobby עם הזנת Network Key]

3.3 שיתוף והורדת קבצים (Dashboard)

- **My Shared Files:** העלאת קבצים לרשת בלחיצה על "Add File".
- **Network Files:** הצגת קבצים זמינים. תג **Swarm** ⚡ מצוין הורדה ממספר מקורות.
- **Active Downloads:** מעקב עם Progress Bar. תמיכה ב-Resume במקרה של ניתוק.

[צילום מסך: Dashboard עם קבצי רשת והורדות פעילות]

פרק 4: מדריך למפתח (הסבר טכני)

4.1 ניהול קבצים (SharedFileManager)

המחלקה מנהלת את פרטי הקבצים וסורקת את תיקיית השיתוף.

```
self.files_metadata
```

: מילון הממפה בין Hash של קובץ לפרטיו.

```
# snippet from shared_files_manager.py
def generate_file_hash(self, file_path):
    sha256_hash = hashlib.sha256()
    with open(file_path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

4.2 גילוי עמיתים (Peer Discovery - peer.py)

מנהל את ה-Networking המבוזר ב-UDP.

```
# snippet from peer.py
def listen_for_peers(self):
    while True:
        data, addr = self.udp_socket.recvfrom(4096)
        try:
            decrypted_data = self.fernet.decrypt(data)
            message = json.loads(decrypted_data.decode())
```

```
# Anti-Replay Attack Check
timestamp = message.get('timestamp')
if abs(time.time() - timestamp) > 30:
    continue

self.update_peer_table(addr[0], message)
except InvalidToken:
    pass
```

4.3 מנגנוני אבטחה והצפנה (security.py)

- **UDP Security:** שימוש ב-AES-128 (Fernet) עם מפתח שנגזר מסיסמת המשתמש.
- **TCP Security:** הצפנה היברידית - RSA להחלפת Session Key ו-AES להעברת הקובץ המהירה.